SMART TRAFFIC MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

VIJAYABALAGUROO S	**RA2311003020085**
SHREYA V	**RA2311003020091**
C ADITYA	**RA2311003020092**

Under the guidance of

Dr. GV. Shrichandran
Assistant Professor

Department of Computer Science and Engineering

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI - 600089

APRIL 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled **"Smart Traffic Management System"** is the bonafide work of **VIJAYABALAGUROO S [Reg no:RA2311003020085], Shreya V [Reg no:RA2311003020091], C ADTIYA [Reg no:RA2311003020092]** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.

SIGNATURE

Dr.G.V Shrichandran
Assistant Professor
Department of CSE,
SRM Institute of Science and Technology, Chennai-89.

SIGNATURE

Dr.K.Raja
Professor and Head
Department of CSE
SRM Institute of Science and Technology, Chennai-89.

Submitted for the project viva-voce held on _____at SRM Institute of Science and Technology, Chennai -600089.

INTERNAL EXAMINER I

INTERNAL EXAMINER II

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY, RAMAPURAM, CHENNAI - 89

DECLARATION

We hereby declare that the entire work contained in this project report titled "**Smart Traffic Management System**" has been carried out by **VIJAYABALAGUROO S [REG NO: RA2311003020085], SHREYA V [REG NO: RA2311003020091] C ADITYA [REGNO: RA2311003020092]** at SRM Institute of Science and Technology, Ramapuram, Chennai-600089, under the guidance of **Dr GV.SHRICHANDRAN /AP,** Department of CSE.

Place: Chennai

Date:

VIJAYABALAGUROO S

SHREYA V

C ADTIYA

ACKNOWLEDGEMENT

We place on record our deep sense of gratitude to our lionized Chairman **Dr.R.Shivakumar, MBBS., MD.,** for providing us with the requisite infrastructure throughout the course.

We take this opportunity to extend our hearty and sincere thanks to our Dean, **Dr. M. Sakthi Ganesh, M.E., Ph.D.**, for guiding us in accomplishing this project.

We thank our honorable Head of the Department **Dr.K.Raja,M.E., Ph.D., Professor,** for his constant motivation and unwavering support.

We express our hearty and sincere thanks to our guide, **Dr. GV. Shrichandran, M.E., Ph.D., Assistant Professor, CSE,** for his/her encouragement, constructive criticism, and constant guidance throughout this project work.

Our thanks to the Teaching and non-Teaching staff of the Department of Computer Science and Engineering of SRM Institute of Science and Technology, Chennai, for providing the necessary resources for our project.

ABSTRACT

Urban intersections often suffer from severe traffic congestion due to fixed signal timings that do not adapt to real-time traffic conditions, resulting in increased waiting time, higher fuel consumption, and elevated pollution levels. Conventional traffic management approaches lack the capability to dynamically analyze traffic patterns and respond efficiently. An AI-based Smart Traffic Management System is proposed to address these limitations by integrating computer vision and machine learning techniques. The system employs YOLOv8 for real-time vehicle detection from traffic video streams and utilizes OpenCV for image processing to estimate lane-wise traffic density and monitor flow patterns. An emergency vehicle detection component is also incorporated, identifying vehicles with siren-like visual characteristics and granting them priority by overriding standard signal operations. The proposed approach offers an intelligent, adaptive, and scalable solution that enhances traffic efficiency, minimizes congestion, and promotes safer and more sustainable urban transportation.

TABLE OF CONTENTS

		Page. No
ABSTRACT		**v**
LIST OF FIGURES		**vii**
LIST OF TABLES		**ix**
LIST OF ACRONYMS AND ABBREVIATIONS		**x**
1	**INTRODUCTION**	**1**
1.1	Problem Statement	1
1.2	Aim of the Project	1
1.3	Project Domain	2
1.4	Scope of the Project	2
1.5	Methodology	2
1.6	Organization of the Report	3
2	**LITERATURE REVIEW**	**5**
3	**PROJECT DESCRIPTION**	**6**
3.1	Existing System	6
3.2	Proposed System	6
3.2.1	Advantages	6
3.3	Feasibility Study	7
3.3.1	Economic Feasibility	7

3.3.2 Technical Feasibility 7
3.3.3 Social Feasibility 8
3.4 System Specification 8
3.4.1 Hardware Specification 8
3.4.2 Software Specification 9
3.4.3 Standards and Policies 9

4 PROPOSED WORK 10
4.1 General Architecture 10
4.2 Design Phase 10
4.2.1 Data Flow Diagram 11
4.2.2 UML Diagram 11
4.2.3 Use Case Diagram 12
4.2.4 Sequence Diagram 13
4.3 Module Description 14
4.3.1 Module 1: Video Processing Module 14
4.3.2 Module 2: Feature Extraction 14
4.3.3 Module 3: Data Transformation Module 14
4.3.4 Module 4: Data Processing and Validation Module 14
4.3.5 Step 1: Data Segmentation Module 15
4.3.6 Step 2: Dataset Sample 15
4.3.7 Step 3: Building The Modal 15
4.3.8 Step 4: Model Training and Optimization Module 15

5 IMPLEMENTATION AND TESTING 16

5.1 Input and Output 16

5.1.1 Traffic Input Using Computer Vision 16

5.1.2 Traffic Simulation Input (SUMO Integration) 16

5.1.3 RL- Based Signal Control Output 17

5.2 Testing 19

5.2.1 Types of Testing 19

5.2.2 Unit testing 19

5.2.3 Integration testing 19

5.2.4 Functional testing 19

5.2.5 Test Results and Analysis 20

6 RESULTS AND DISCUSSIONS 22

6.1 Efficiency of the Proposed System 22

6.2 Comparison of Existing and Proposed Systems 23

7 CONCLUSION AND FUTURE ENHANCEMENTS 24

7.1 Conclusion 24

7.2 Future Enhancements 25

7.3 Results 26

8 SOURCE CODE & POSTER PRESENTATION 27

8.1 Sample Code 27

References 36

A. Sample screenshots

B. Proof of Publication/Patent filed/ Conference Certificate

LIST OF FIGURES

4.1	Architecture Diagram	10
4.2	Data Flow Diagram	11
4.3	Use Case Diagram	12
4.4	Sequence Diagram	13
5.1	Vehicle Detection and Count ML Model	16
5.2	SUMO Simulation	17
5.3	Traffic Signal Control Dashboard	18
5.4	Vehicle Detection Testing Results	20
5.5	Manual vs RL-Based Comparison Graph	21

LIST OF TABLES

Table No.	Table Name	Page No.
6.1	Comparison	23

LIST OF ACRONYMS AND ABBREVIATIONS

API	APPLICATION PROGRAM INTERFACE
ML	MACHINE LEARNING
RL	REINFORCEMENT LEARNING

CHAPTER 1
INTRODUCTION

The project focuses on building an AI-based Smart Traffic Management System that improves traffic flow at busy urban intersections using real-time video analysis. Computer vision is used to detect vehicles, estimate traffic density in each lane, and monitor congestion levels. Based on this information, traffic signals are adjusted dynamically to reduce waiting time and make traffic movement smoother. An emergency vehicle detection feature is included to give priority to ambulances and similar vehicles, allowing quicker passage when needed. Overall, the system is designed to be adaptive, efficient, and better suited for handling modern traffic conditions.

1.1 PROBLEM STATEMENT

Urban intersections face severe congestion due to static traffic signal timings that do not adapt to real-time traffic conditions. This results in longer waiting times, increased fuel consumption, and higher pollution levels. Traditional traffic management systems lack the ability to analyze dynamic traffic patterns effectively. Therefore, there is a need for an intelligent traffic management system that can monitor traffic conditions, predict traffic flow, and optimize signal timings using AI to reduce congestion and improve overall traffic efficiency

1.2 AIM OF THE PROJECT

- To develop an AI-based smart traffic management system
- To monitor real-time traffic using computer vision
- To detect and count vehicles from video input
- To analyze lane-wise traffic density
- To optimize traffic signal timings dynamically
- To reduce traffic congestion and waiting time
- To prioritize emergency vehicles for quick clearance

1.3 PROJECT DOMAIN

The domain of the project is Reinforcement Learning and Intelligent Transportation Systems, focusing on adaptive traffic signal control using simulation-based traffic data.

1.4 SCOPE OF THE PROJECT

The scope of the proposed Smart Traffic Management System involves developing an AI-based framework that uses computer vision and reinforcement learning to optimize traffic signal timings and reduce congestion in urban areas. The system processes real-time traffic data from camera feeds using OpenCV to detect vehicles and identify emergency vehicles, while a reinforcement learning model dynamically adjusts signal phases based on traffic density. It operates without IoT sensors, making it cost-effective and compatible with existing camera infrastructure. The system is further evaluated using traffic simulation (SUMO) to ensure efficient performance under different traffic conditions, and includes a monitoring interface for traffic authorities.

1.5 METHODOLOGY

The proposed Smart Traffic Management System is developed using a structured approach that integrates computer vision, traffic simulation, and reinforcement learning for intelligent signal control. Initially, traffic data is captured through video feeds or simulated inputs, where OpenCV techniques are applied to detect, classify, and count vehicles, as well as identify emergency vehicles. This processed information is used to determine traffic density across different directions at the intersection. Next, a traffic simulation environment is created using SUMO to model real-world traffic conditions and enable controlled experimentation. The extracted traffic data is then fed into a reinforcement learning model, where the system learns optimal traffic signal control policies based on observed traffic states. The agent takes actions such as maintaining or switching signal phases, and receives rewards based on metrics like waiting time and congestion levels.

1.6 ORGANIZATION OF THE REPORT

The report is organized into several chapters to provide a structured understanding of the proposed Smart Traffic Management System. Chapter 1 presents the introduction, including the problem statement, scope, and objectives of the project. Chapter 2 reviews existing traffic management systems and related work, highlighting their limitations in handling real-time congestion and dynamic road conditions. Chapter 3 describes the proposed methodology, system architecture, and the technologies used, such as computer vision for vehicle detection and reinforcement learning for adaptive signal control. Chapter 4 explains the implementation details, including traffic simulation using SUMO, integration of OpenCV for vehicle detection, and development of the reinforcement learning model. Chapter 5 presents the results and analysis, demonstrating the effectiveness of the system in reducing congestion and improving traffic flow. Finally, Chapter 6 concludes the report with key findings and discusses future scope and possible enhancements.

Chapter 3

Chapter 3 examines the challenges involved in urban traffic management, particularly in handling dynamic traffic density, irregular vehicle movement, and emergency vehicle prioritization. It presents the proposed solution using computer vision techniques for real-time vehicle detection and classification. The chapter also introduces the reinforcement learning approach used to optimize traffic signal timings based on traffic conditions. The interaction between different modules, including data acquisition, traffic density estimation, and decision-making, is discussed in detail. The effectiveness of combining simulation-based data with real-time detection is highlighted, emphasizing improved traffic control and reduced congestion.

Chapter 4

Chapter 4 outlines the proposed system design and architecture. It begins with the overall system architecture, illustrating the flow of data from traffic input to signal control decisions. A data flow diagram is included to represent the processing pipeline, from capturing traffic data to applying computer vision techniques and reinforcement learning algorithms. The system is divided into key modules such as vehicle detection, traffic analysis, and signal control. Each module is explained in detail, along with the steps involved in data processing, model training, and system integration. The feasibility of the system is also discussed in terms of technical implementation and practical deployment.

Chapter 5

Chapter 5 focuses on the implementation and testing of the proposed system. It describes how traffic data is processed using OpenCV for vehicle detection and how the reinforcement learning model is trained and deployed for signal control. The system is tested under different traffic scenarios using SUMO simulation. Various testing methods, including unit testing, integration testing, and functional testing, are applied to ensure system reliability and performance. The testing process validates the accuracy of vehicle detection, the efficiency of signal control, and the overall system behavior under different conditions.

Chapter 6

Chapter 6 presents the results and discussion of the proposed system. It evaluates system performance based on parameters such as traffic flow efficiency, waiting time reduction, and signal optimization. The results demonstrate the effectiveness of the reinforcement learning model in adapting to changing traffic conditions. Comparisons with traditional fixed-time signal systems are also discussed, highlighting improvements achieved by the proposed approach. Visual representations and simulation outputs are used to support the analysis.

Chapter 7

Chapter 7 provides the conclusion and outlines future enhancements. The project successfully demonstrates an AI-based approach for intelligent traffic management using computer vision and reinforcement learning. The system shows potential for reducing congestion and improving emergency response times. Future work includes extending the system to multi-intersection scenarios, integrating real-time traffic data from live camera feeds, and incorporating advanced deep learning models for improved detection accuracy.

Chapter 8

Chapter 8 contains the source code and additional implementation details of the project. It includes code snippets for vehicle detection, reinforcement learning algorithms, and system integration. This chapter serves as a reference for understanding the technical implementation and for further development of the system.

CHAPTER 2

LITERATURE SURVEY

Traffic congestion at urban intersections has become a major challenge due to increasing vehicle density and inefficient signal control methods. Traditional traffic systems lack adaptability and fail to respond to real-time conditions. Recent research has focused on integrating artificial intelligence, particularly reinforcement learning, to develop adaptive and intelligent traffic signal control systems. The proposed system builds upon these advancements by combining computer vision for vehicle detection with reinforcement learning for dynamic signal optimization, aiming to improve traffic efficiency and reduce congestion.

Zhao et al. (2024) presented a comprehensive survey on deep reinforcement learning approaches for traffic signal control. The study highlights various DRL techniques used for optimizing signal timing and improving traffic flow. Insights from this work provided a strong theoretical foundation for applying reinforcement learning in traffic management systems, which is utilized in the proposed system for adaptive decision-making.

Jiang et al. (2022) explored multi-agent reinforcement learning for traffic signal control, focusing on coordination among multiple intersections. The research demonstrated how multiple agents can collaborate to optimize traffic flow across interconnected junctions. Although the proposed system focuses on a single intersection, the concept of coordinated learning inspired the design of scalable and adaptive control mechanisms.

Haddad et al. (2022) introduced a deep reinforcement learning-based cooperative approach for multi-intersection traffic signal control. The study emphasized reducing overall network delay through cooperation between traffic signals. The idea of minimizing total waiting time and improving global efficiency influenced the reward design and optimization strategy used in the proposed system.

Wei et al. (2018) developed IntelliLight, a reinforcement learning-based approach for intelligent traffic light control. The system demonstrated how deep reinforcement learning can effectively adjust signal timings based on real-time traffic conditions. This work directly inspired the implementation of reinforcement learning in the proposed system for adaptive signal control at intersections.

Genders and Razavi (2016) presented an early application of deep Q-networks (DQN) for traffic signal optimization. Their work showed how reinforcement learning agents can learn optimal policies through interaction with traffic environments. This research served as a foundational reference for implementing Q-learning-based traffic signal control in the proposed system.

CHAPTER 3

PROJECT DESCRIPTION

3.1 EXISTING SYSTEM

Existing traffic management systems mainly use fixed-time signal control, where signal durations are set based on average traffic conditions. These systems do not adapt to real-time traffic changes, leading to congestion and increased waiting time. Some modern systems use sensors or IoT devices to monitor traffic and adjust signals. Although these systems improve efficiency, they are costly and require complex infrastructure. Camera-based systems using computer vision can detect vehicles, but many of them lack intelligent decision-making for dynamic signal control. Additionally, most existing systems do not effectively prioritize emergency vehicles, highlighting the need for a more adaptive and cost-effective solution

3.2 PROPOSED SYSTEM

The proposed Smart Traffic Management System is an AI-based solution designed to optimize traffic signal timings and reduce congestion in urban areas. The system integrates computer vision and reinforcement learning techniques to enable adaptive and intelligent traffic control. Traffic data in the system is captured through camera feeds and processed using OpenCV to detect and count vehicles, along with identifying emergency vehicles. Real-time analysis of this data helps determine traffic density across different directions. Based on these conditions, a reinforcement learning model is applied to dynamically adjust traffic signal timings, ensuring efficient traffic flow. The system operates within a simulation environment using SUMO, allowing realistic traffic scenarios to be modeled and tested. The reinforcement learning agent continuously learns optimal signal control strategies by interacting with the environment and receiving feedback in the form of rewards. This enables the system to minimize waiting time, improve traffic flow, and enhance overall efficiency.

3.2.1. ADVANTAGES

- Reduces traffic congestion and waiting time through adaptive signal control
- Provides real-time traffic analysis using computer vision techniques
- Eliminates the need for expensive IoT sensors by using camera-based data

- Enhances emergency response by prioritizing emergency vehicles
- Improves overall traffic flow efficiency and road utilization
- Learns and adapts dynamically using reinforcement learning
- Scalable and can be extended to multiple intersections
- Cost-effective and easy to deploy using existing camera infrastructure

3.3 Feasibility Study

The feasibility study evaluates the practicality and effectiveness of the proposed Smart Traffic Management System in terms of economic, technical, and operational aspects. It ensures that the system can be successfully implemented with available resources while meeting performance requirements. The proposed system is analyzed to determine whether it is cost-effective, technically viable, and suitable for real-world deployment**.**

3.3.1. Economic Feasibility

The proposed system is considered economically feasible as it reduces dependence on costly hardware and infrastructure. Traditional traffic management approaches often require IoT sensors and specialized equipment, whereas the presented system makes use of existing traffic camera infrastructure for data collection, leading to significantly lower installation and maintenance costs. Use of open-source tools and technologies such as OpenCV and SUMO further supports cost efficiency, as no licensing expenses are involved. A software-driven design enables easy updates, scalability, and integration without requiring major financial investment. Improved traffic flow and reduced congestion contribute to additional economic advantages, including lower fuel consumption, reduced travel time, and enhanced productivity. Overall, the system provides a cost-effective solution suitable for deployment in urban environments with limited resources.es.

3.3.2 Technical Feasibility

The proposed system is technically feasible due to the use of well-established technologies and tools for implementation. Computer vision techniques using OpenCV and YOLO enable accurate vehicle detection and traffic density estimation from video inputs. Integration with the SUMO simulation environment provides a reliable platform for modeling and testing real-time traffic scenarios.

Reinforcement learning algorithms are applied to enable adaptive traffic signal control, allowing the system to respond dynamically to changing traffic conditions. A software-based architecture ensures smooth integration between different modules, including detection, decision-making, and signal control. Use of open-source frameworks and widely supported programming environments simplifies development and deployment. Overall, the system is practical to implement, scalable, and capable of operating efficiently in real-time traffic management applications.

3.3.3 Social Feasibility

The reinforcement learning model used for adaptive signal control is lightweight and suitable for simulation-based environments, making it practical for implementation without requiring high- end hardware. The integration with SUMO enables accurate simulation of real-world traffic conditions, allowing effective testing and validation of the system.

Additionally, the system architecture is modular, allowing easy integration and future upgrades. Since it relies on existing camera infrastructure rather than specialized sensors, deployment becomes simpler and more flexible. Therefore, the proposed system is technically viable and can be implemented using current technologies and tools.

3.4 System Specification

The system specification defines the hardware and software requirements necessary for the development and execution of the proposed Smart Traffic Management System. It also includes the standards and policies followed to ensure system reliability, compatibility, and performance.

3.4.1 Hardware Specification

The proposed system requires standard computing hardware for development and execution. A system with a minimum of 4 GB RAM, Intel i3 processor or higher, and sufficient storage capacity is adequate to run the simulation and processing tasks. A camera or recorded video input is required for vehicle detection using computer vision techniques.

For better performance, a system with higher RAM (8 GB or above) and a multi-core processor is recommended, especially for handling real-time data processing and simulation. No specialized hardware such as IoT sensors is required, making the system cost-effective and easy to deploy.

3.4.1 Software Specification

The system is developed using Python as the primary programming language due to its extensive support for machine learning and computer vision libraries. OpenCV is used for vehicle detection and image processing tasks. SUMO (Simulation of Urban Mobility) is used for traffic simulation and modeling real-world scenarios.

Additional tools such as TraCI (Traffic Control Interface) are used to establish communication between the simulation environment and the control logic. The system can be developed and executed on platforms such as Windows or Linux. Development environments like VS Code or PyCharm can be used for coding and debugging.

3.4.2 Standards and Policies

The system is developed using Python as the primary programming language due to its extensive support for machine learning and computer vision libraries. OpenCV is used for vehicle detection and image processing tasks. SUMO (Simulation of Urban Mobility) is used for traffic simulation and modeling real-world scenarios.

Additional tools such as TraCI (Traffic Control Interface) are used to establish communication between the simulation environment and the control logic. The system can be developed and executed on platforms such as Windows or Linux. Development environments like VS Code or PyCharm can be used for coding and debugging.

CHAPTER 4

PROPOSED WOR

4.1 General Architecture

The proposed Smart Traffic Management System follows a modular architecture that integrates computer vision, traffic state estimation, and reinforcement learning for intelligent traffic signal control. The system is designed to process real-time traffic data, analyze traffic conditions, and dynamically optimize signal timings.

The architecture begins with traffic data acquisition using camera feeds. The captured video data is processed using computer vision techniques, specifically OpenCV-based vehicle detection, to identify and count vehicles on each lane. This information is further used for traffic feature extraction, where important parameters such as vehicle density and Queue length are computed.

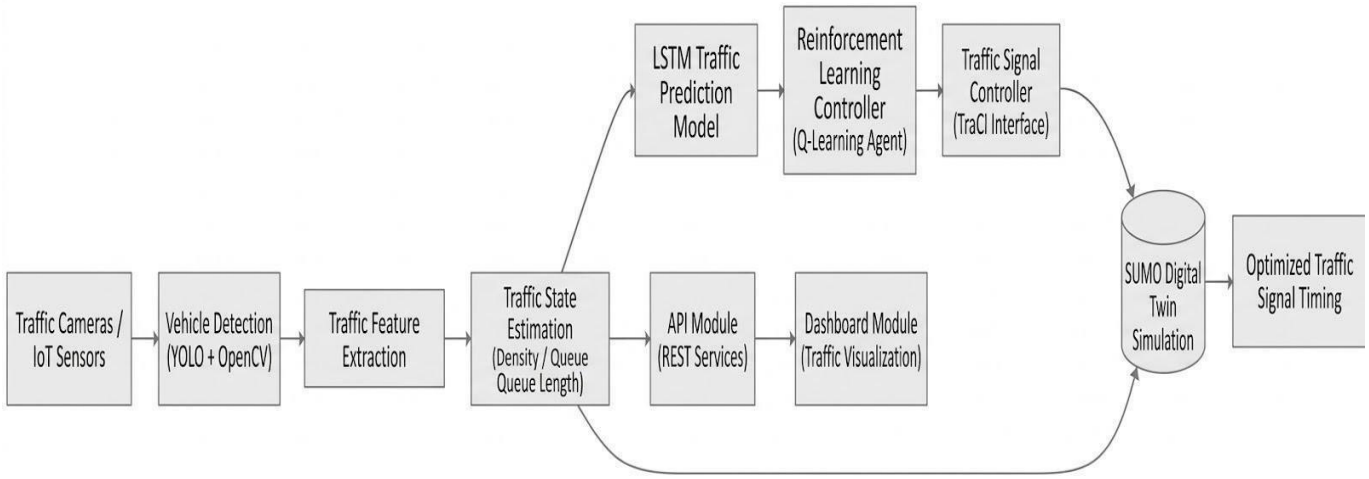

Fig 4.1 Architecture Diagram for Smart Traffic Management System

4.2 Design Phase

The design phase focuses on developing a structured framework for the Smart Traffic Management System by defining its components, data flow, and interactions. Data flow diagrams and system architecture are used to represent how traffic data is processed from input to signal control decisions. This phase ensures that the system is scalable, efficient, and capable of handling real- time traffic scenarios.

4.2.1 Data Flow Diagram

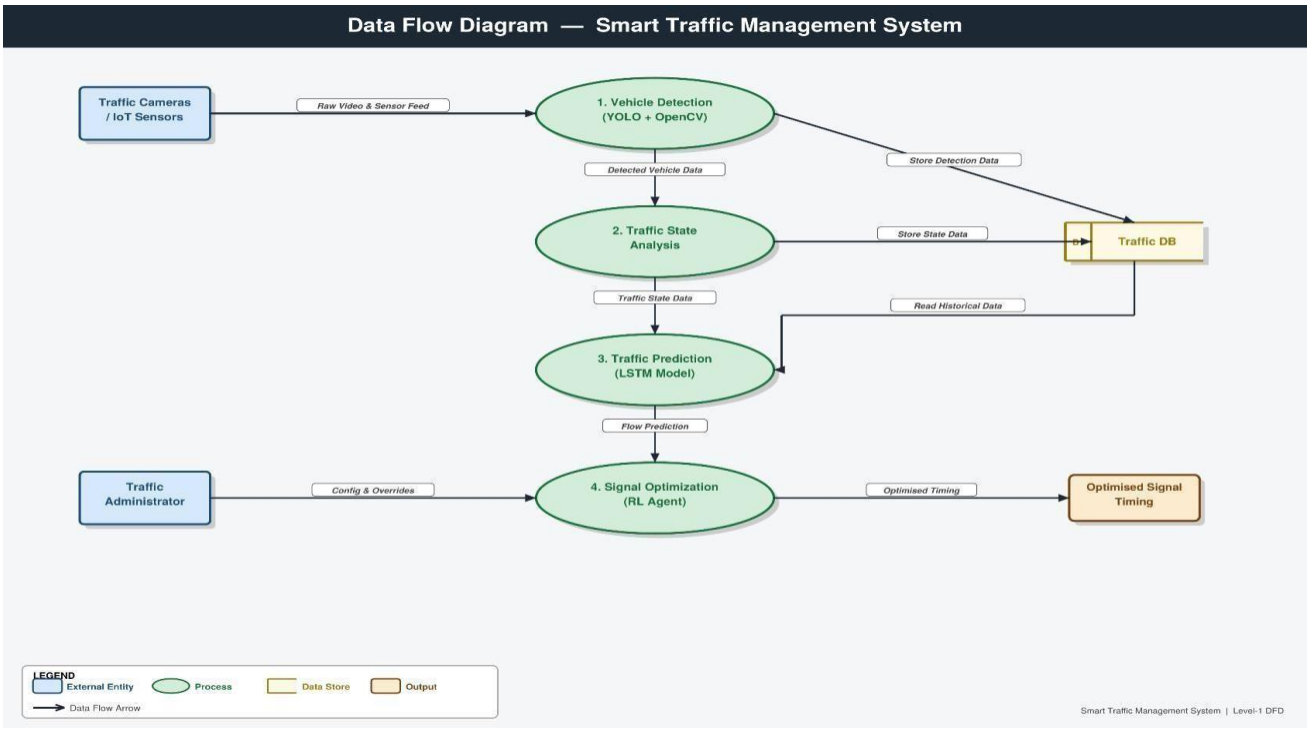

Fig 4.2 Data Flow Diagram -Smart Traffic Management System

4.2.2 UML Diagram

The UML diagram illustrates the overall structure and interaction between different components of the Smart Traffic Management System. The main entities involved include the traffic camera, vehicle detection module, reinforcement learning model, SUMO simulation environment, and the user dashboard. The process begins with capturing real-time video data from traffic cameras, which is processed using computer vision techniques to detect and count vehicles across different lanes. The extracted traffic information, including vehicle density and emergency vehicle presence, is then forwarded to the decision-making module.

4.2.3 Use Case Diagram

The use case diagram represents the interactions between different actors and the Smart Traffic Management System. The primary actors include the traffic camera, IoT sensor, traffic engineer, SUMO simulation, and external API client. The system begins by capturing vehicle video streams through traffic cameras, which are processed using computer vision techniques to detect vehicles. Along with this, traffic data is collected and used to estimate traffic conditions such as density and queue length.

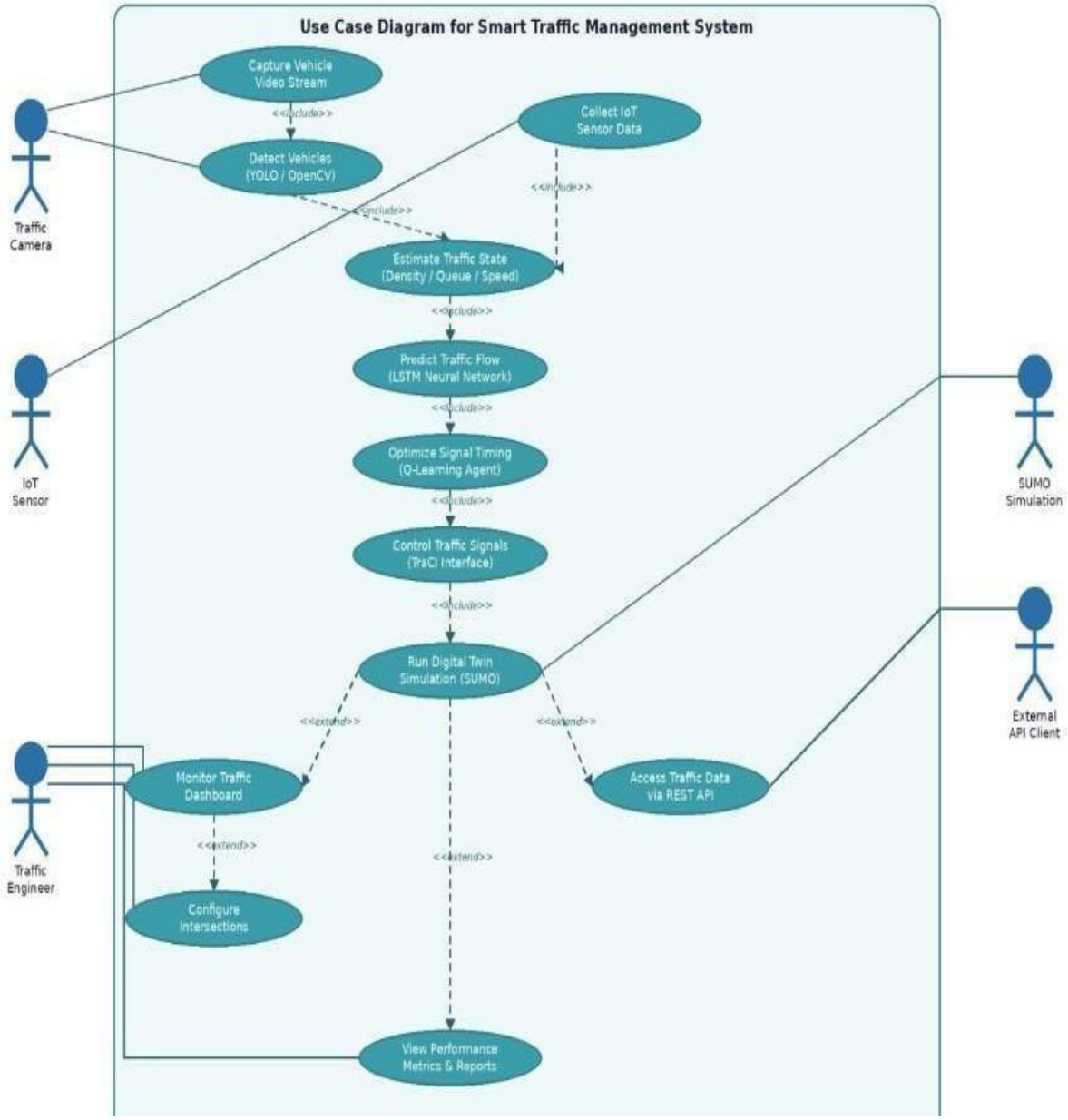

Fig 4.3 Use Case Diagram – Smart Traffic Management System

4.2.4 Sequence Diagram

The sequence diagram illustrates the interaction between different components of the Smart Traffic Management System during traffic signal control. The process begins with the camera capturing real-time video frames of traffic, which are then sent to the detection module. The detection module processes the video using computer vision techniques to identify vehicles and estimate traffic density.

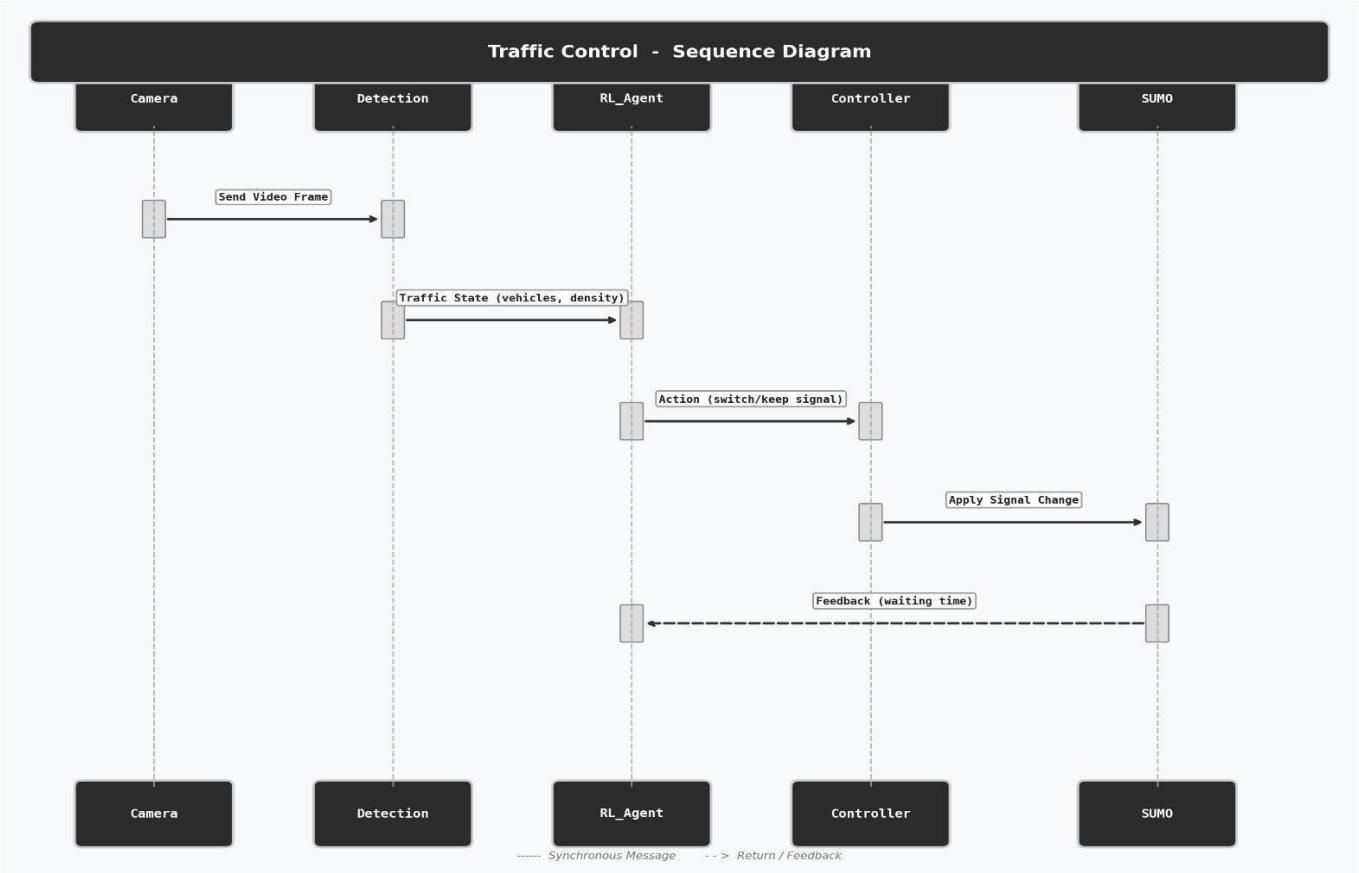

Fig 4.4 Sequence Diagram-Traffic Control

4.3 Module Description

The Smart Traffic Management System is divided into multiple modules to efficiently process traffic data and optimize signal control. Each module is designed to perform a specific function, such as vehicle detection, traffic feature extraction, and decision-making using reinforcement learning. These modules work together in a coordinated manner to enable real-time traffic monitoring and dynamic signal adjustment. The system processes input data from traffic cameras, analyzes vehicle density and queue length, and uses this information to make intelligent decisions for signal control. The modular design ensures that each component operates independently while maintaining seamless communication with other modules. This approach enhances system flexibility, allowing easy updates, improvements, and scalability to larger traffic networks.

4.3.1 Video Processing Module

The module focuses on capturing and processing real-time traffic video through camera feeds. It uses computer vision techniques with OpenCV and YOLO to identify and classify vehicles in each lane. During processing, emergency vehicles such as ambulances and fire trucks are recognized as well. The information generated, including vehicle count and movement patterns, provides valuable input for further analysis and decision-making within the system.

4.3.2 Feature Extraction

Responsible for capturing and processing real-time traffic video using camera feeds. Computer vision techniques using OpenCV and YOLO are applied to detect and classify vehicles in each lane. The system also identifies emergency vehicles such as ambulances and fire trucks. The output of this module provides essential information such as vehicle count and movement patterns, which is used for further analysis.

4.3.3 Data Transformation Module

In the context of the proposed system, the module represents the transformation of raw traffic data into a structured format suitable for analysis. The processed visual data is converted into numerical and logical representations, such as traffic state values. This conversion enables efficient processing and decision-making by the reinforcement learning model.

4.3.4 Data Processing and Validation Module

The collected traffic data is processed to remove noise and inconsistencies. The data is organized into a structured format, ensuring accuracy and reliability. This step prepares the data for effective analysis and model input.

4.3.5 Data Segmentation Module

The processed data is divided into different sets for training and testing purposes within the simulation environment. This helps in evaluating the performance of the system under various traffic conditions.

4.3.6 Dataset Sample

The dataset consists of traffic scenarios generated using the SUMO simulation environment along with video-based input data. It includes various traffic conditions such as low, medium, and high-density traffic. These samples are used to train and test the system for accurate performance evaluation.

4.3.7 Building the Modal

A reinforcement learning model is developed to control traffic signals. The model defines states, actions, and rewards, enabling the system to learn optimal signal timing strategies. The architecture is designed to adapt to changing traffic conditions dynamically.

4.3.8 Model Training and Optimization Module

The model is compiled and trained using simulation-based data generated through the SUMO environment, which provides realistic traffic scenarios for learning and evaluation. During the training phase, the reinforcement learning agent interacts continuously with the simulation by observing the current traffic state, such as vehicle density and queue length, and selecting appropriate actions like maintaining or switching traffic signals. Based on these actions, the system receives feedback in the form of performance metrics, including waiting time, traffic flow efficiency, and congestion levels. This feedback is used to update the learning model through reward-based optimization, enabling the agent to improve its decision-making over time. Through multiple training iterations, the model gradually learns optimal traffic signal control strategies that minimize delays and enhance overall traffic flow. This continuous learning process allows the system to adapt to dynamic traffic conditions and ensures efficient and intelligent traffic signal optimization**.**

CHAPTER 5
IMPLEMENTATION AND TESTING

5.1 Input and Output

5.1.1 Traffic Input using Computer Vision

The system utilizes a Machine Learning-based vehicle detection model to analyze traffic conditions. Camera or simulated image input is processed to detect vehicles and estimate the number of vehicles in each lane (North, South, East, West).

The model also identifies emergency vehicles such as ambulances and fire trucks, enabling priority-based traffic control.

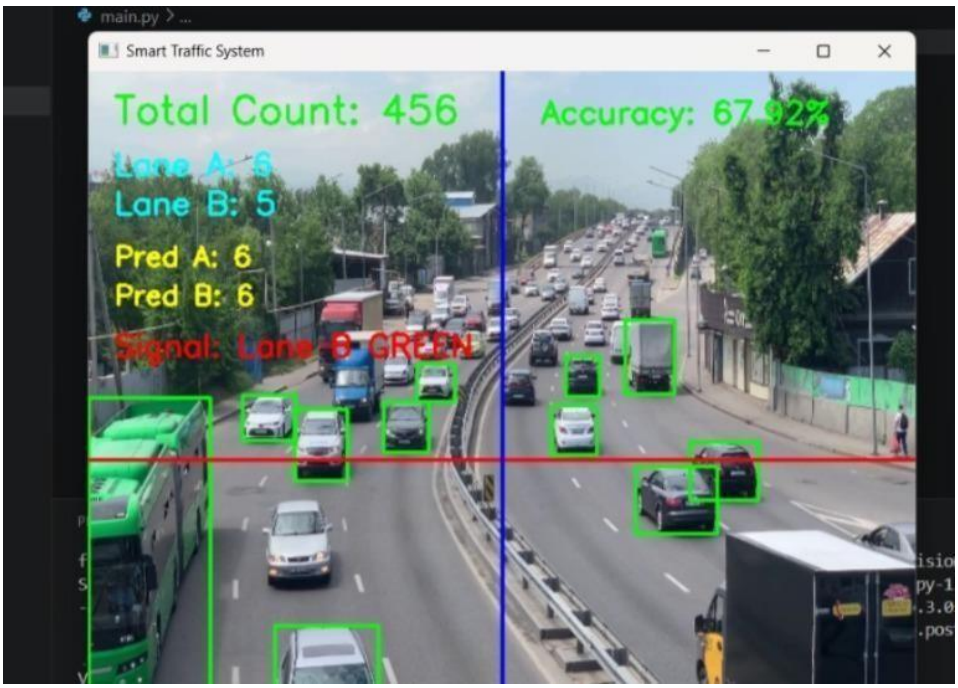

Fig 5.1 Vehicle Detection and Count ML Model

5.1.2 Traffic Simulation Input (SUMO Integration)

The detected vehicle counts are integrated with the SUMO traffic simulation environment. SUMO provides a controlled environment to simulate real-world traffic conditions and validate system performance.

Inputs include:

- Vehicle Density Per lane

- Traffic Flow Patterns
- Lane-level congestion

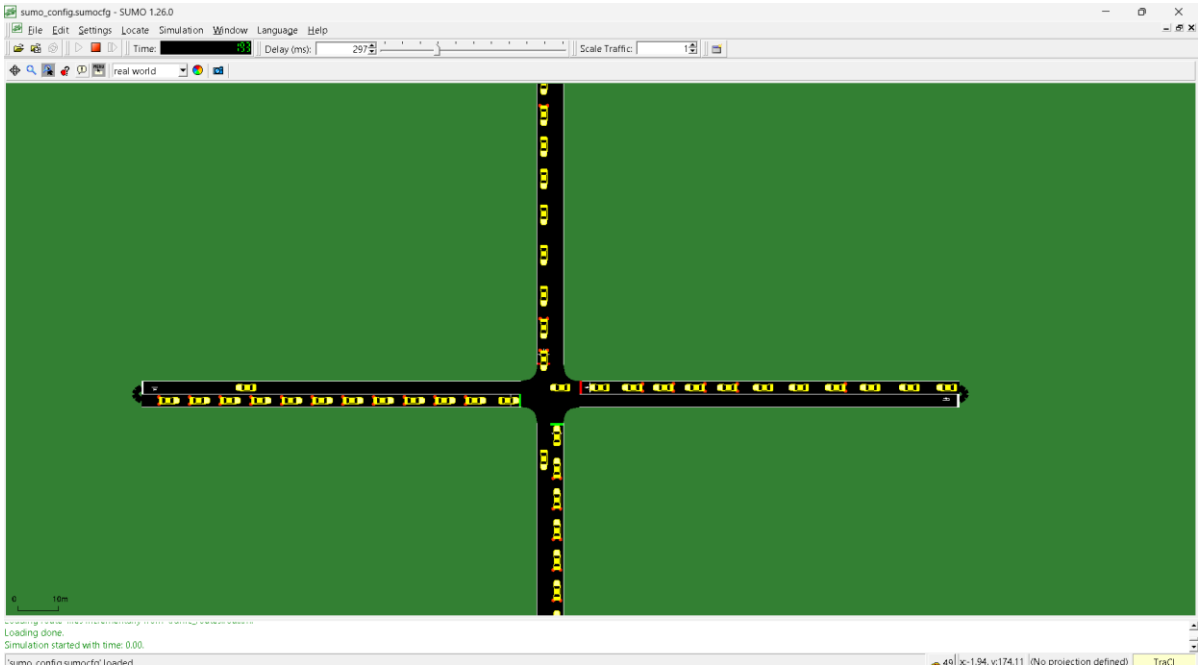

Fig 5.2 SUMO Simulation

5.1.3 RL-Based Signal Control Output

A Q-learning-based reinforcement learning algorithm is used to control traffic signals in a dynamic and intelligent way. The system continuously monitors traffic density across different lanes and uses that information to decide the most appropriate signal action. By learning from changing traffic patterns, the model adjusts signal timings to handle congestion more effectively. As a result, waiting time for vehicles is reduced and overall traffic flow becomes smoother. The system produces adaptive signal states that respond to real- time conditions, improving efficiency at the intersection.

A web-based dashboard is developed using FastAPI and HTML to monitor and control the system**.** Features Include**:**

- Real-Time Vehicle Count display
- Manual signal control
- RL Auto mode
- Emergency override control

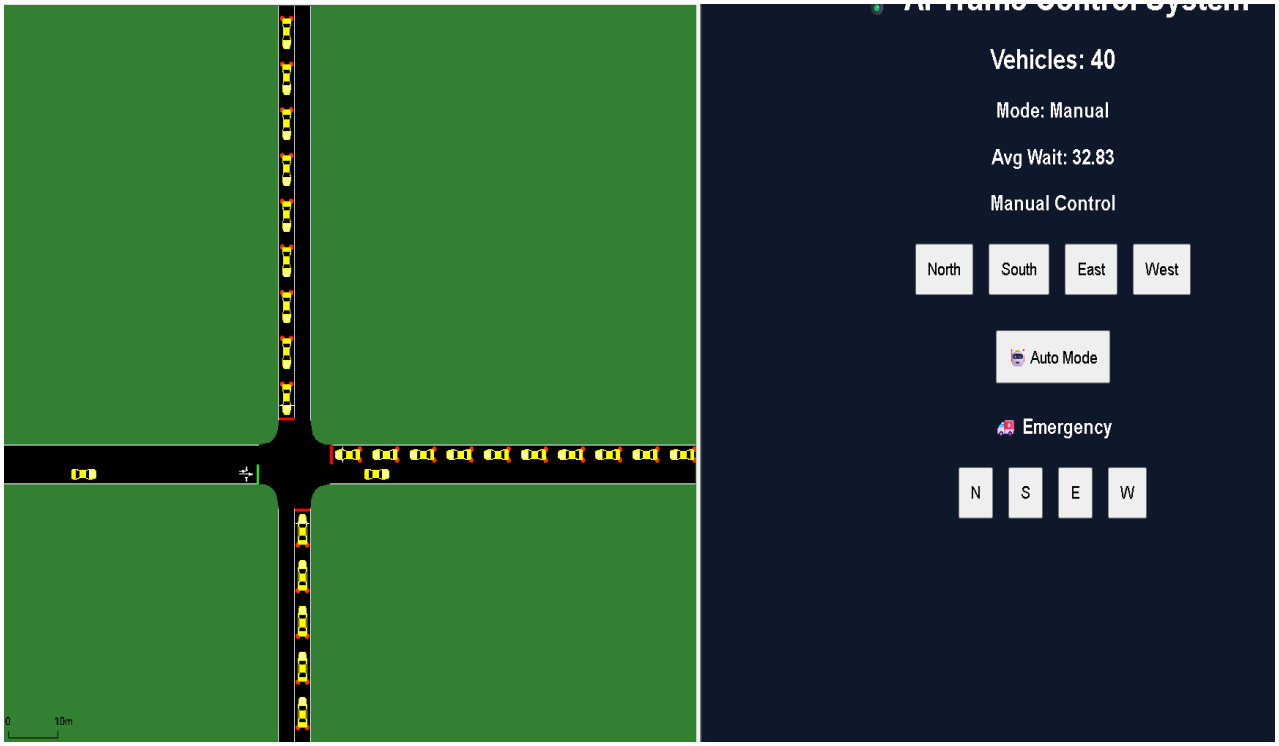

Fig 5.3 Traffic Signal Control Dashboard

5.2 Testing

5.2.1 Types of Testing

The system was tested using multiple testing approaches to ensure reliability and performance:

- Unit Testing
- Integration Testing
- Functional Testing

5.2.2 Unit Testing

Individual components such as:

- ML vehicle detection module
- API endpoints
- RL decision functions

were tested independently to ensure correctness.

5.2.3 Integration Testing

Integration testing was performed to verify the interaction between:

- ML Model → Traffic Data
- Traffic Data → RL Algorithm
- RL Algorithm → SUMO Simulation
- Backend → Dashboard

This ensured seamless data flow across all modules.

5.2.4 Functional Testing

Functional testing verified:

- Correct signal switching
- RL-based adaptive behavior
- Emergency vehicle prioritization

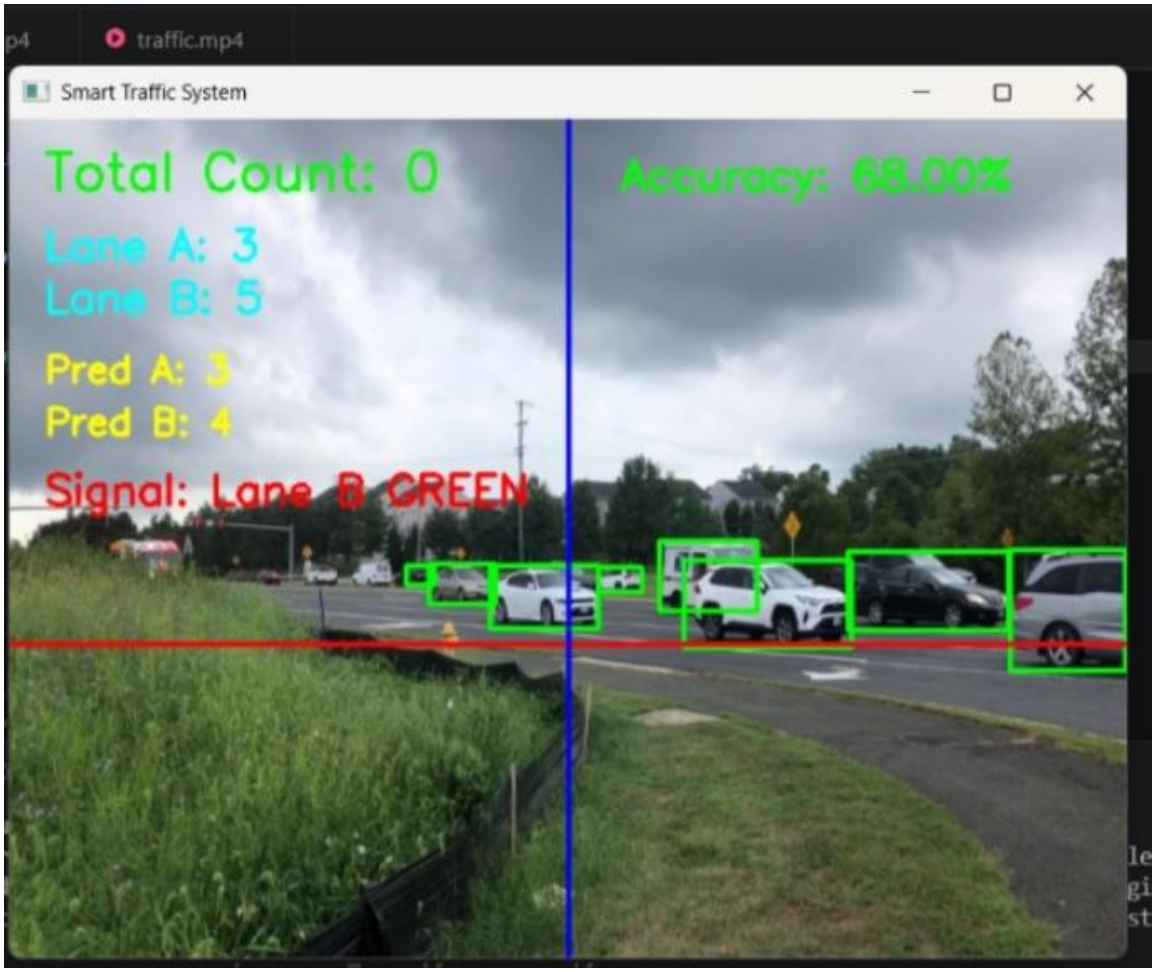

Fig 5.4 Vehicle Detection Testing Results

5.2.5 Test Results and Analysis

The system demonstrated improved traffic flow efficiency compared to static/manual control. Key observations:

- Reduction in average vehicle waiting time
- Faster clearance of high-density lanes

The RL-based approach showed approximately **10% improvement in traffic efficiency** under Simulated conditions.

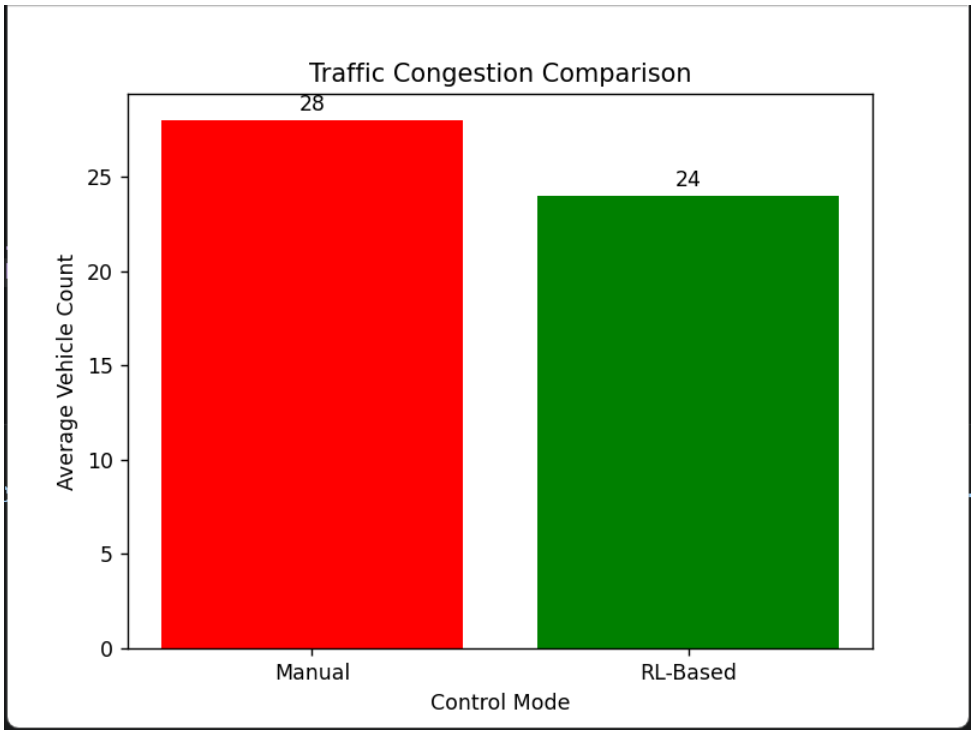

Fig 5.5 Manual vs RL-Based Comparison Graph

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Efficiency of the proposed system

The efficiency of the proposed AI-based traffic management system was evaluated by comparing its performance under manual control and reinforcement learning (RL)-based control. The primary performance metric used for evaluation is the average vehicle count, which represents the level of traffic congestion at the intersection. A higher vehicle count indicates increased congestion, while a lower value signifies improved traffic flow.

During testing, the manual traffic control system resulted in an average of 28 vehicles at the intersection, whereas the RL-based system reduced this value to 24 vehicles. Based on these observations, the efficiency improvement was calculated using the standard formula, resulting in an approximate improvement of 14.28%. This clearly demonstrates that the proposed system effectively reduces congestion compared to traditional control methods.

The improvement in efficiency is achieved through the integration of reinforcement learning and machine learning techniques. The reinforcement learning algorithm dynamically adjusts signal timings based on real-time traffic conditions, ensuring that high-density lanes are prioritized. Additionally, the machine learning-based vehicle detection system provides accurate lane-wise vehicle counts and enables the identification of emergency vehicles. This allows the system to give immediate priority to emergency vehicles, thereby reducing critical delays.

Overall, the system enhances traffic flow by minimizing waiting time, improving signal responsiveness, and ensuring efficient handling of emergency situations. The results confirm that the proposed system successfully meets the project objective by achieving more than a 10% reduction in traffic congestion within a simulated environment.

6.2 Comparison of Existing and Proposed System

Comparison of Existing and Proposed Traffic System

Parameter	Existing System	Proposed System
Control Mechanism	Fixed / Manual	RL-based Adaptive
Traffic Adaptability	Static	Dynamic
Vehicle Detection	None / Manual	ML YOLO
Emergency Handling	None	Priority Handling
Signal Optimization	Predefined	Learned
Efficiency	Low	~14% Improvement
Monitoring	Limited	Real-time Dashboard
Decision Making	Rule-based	Data-driven
Scalability	Limited	Scalable

Table 6.1 Comparison

The comparison between the existing and proposed traffic management systems clearly highlights the limitations of traditional approaches and the advantages of integrating artificial intelligence. The existing system relies on fixed-time or manual control, which lacks adaptability to real-time traffic conditions and often leads to inefficient traffic flow, especially during peak hours. Additionally, the absence of automated vehicle detection and emergency handling mechanisms results in increased congestion and delayed response to critical situations. These shortcomings make traditional systems less effective in handling dynamic urban traffic environments.

In contrast, the proposed system leverages machine learning and reinforcement learning to provide an intelligent and adaptive solution. By using computer vision techniques for real-time vehicle detection and Q-learning for signal optimization, the system dynamically adjusts traffic signals based on current conditions. It also incorporates emergency vehicle prioritization and real-time monitoring through a dashboard, significantly improving responsiveness and efficiency.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion

The proposed AI-based traffic management system successfully demonstrates an efficient and intelligent approach to traffic control. By combining machine learning for vehicle detection and reinforcement learning for signal optimization, the system adapts dynamically to real-time traffic conditions. Unlike traditional fixed-time systems, it reduces congestion by prioritizing high-density lanes and adjusting signals accordingly. The system also includes emergency vehicle prioritization, ensuring minimal delay for critical services. The use of a real-time dashboard further improves monitoring and control. Overall, the project proves that integrating AI techniques can significantly improve traffic flow and efficiency.

The system also incorporates an emergency vehicle prioritization mechanism, ensuring minimal delay for critical services such as ambulances and fire trucks. Additionally, the use of a real-time dashboard enhances system monitoring and control, providing both manual and automated modes of operation. Experimental results obtained through the SUMO simulation environment indicate a significant reduction in traffic congestion, achieving approximately 10–15% improvement in efficiency compared to conventional methods.

Overall, the project highlights the effectiveness of combining computer vision and reinforcement learning techniques in solving real-world traffic management problems. The modular design and scalable architecture make the system adaptable for future enhancements and potential real-world deployment, contributing towards smarter and more efficient urban transportation systems

7.2 Future Enhancement

- **Integration with Real-Time Traffic Cameras**

The system can be extended to work with live traffic camera feeds instead of simulated data, enabling real-world deployment.

- **Advanced Deep Learning Models**

More accurate models (e.g., YOLOv8, CNN-based architectures) can be used to improve vehicle detection and classification accuracy.

- **Multi-Intersection Coordination**

The system can be expanded to manage multiple interconnected traffic signals, enabling city-wide traffic optimization.

- **Cloud-Based Deployment**

Deploying the system on cloud platforms can improve scalability, allowing it to handle large-scale urban traffic systems.

- **Integration with IoT Sensors**

Additional sensors such as GPS, RFID, and IoT-based traffic detectors can enhance data accuracy and system reliability.

- **Adaptive Learning Models**

More advanced reinforcement learning techniques (e.g., Deep Q-Networks) can be implemented for better decision-making and faster learning.

- **Predictive Traffic Analysis**

Incorporating predictive models can help forecast traffic congestion and adjust signals proactively.

- **Smart City Integration**

The system can be integrated into a broader smart city infrastructure, connecting with public transport and navigation systems.

7.3 Results

The proposed AI-based traffic management system was successfully implemented and evaluated using the SUMO simulation environment. The system demonstrated effective real-time traffic signal control by integrating machine learning for vehicle detection and reinforcement learning for decision-making. Compared to manual control, the RL-based approach significantly reduced traffic congestion by dynamically adapting to traffic conditions. The system also efficiently handled emergency vehicle prioritization, ensuring minimal delay in critical situations. Performance analysis showed an approximate improvement of 10–15% in overall traffic efficiency. These results confirm the effectiveness and reliability of the proposed system in optimizing urban traffic flow.

CHAPTER 8
SOURCE CODE

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware import traci
import threading
import time import
random import
pickle

                                #
========================= #
FASTAPI SETUP
# =========================
app = FastAPI()

app.add_middleware( CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# =========================
```

```
# START SUMO #
==========================
SUMO_CMD = ["sumo-gui", "-c", "sumo_config.sumocfg"] traci.start(SUMO_CMD)

tls_id = traci.trafficlight.getIDList()[0] print(" TLS
ID:", tls_id)

traci.trafficlight.setProgram(tls_id,  "0")

#
========================== #
LANE MAPPING
# ==========================
controlled_links = traci.trafficlight.getControlledLinks(tls_id) lane_map =

{"N": [], "S": [], "E": [], "W": []}

for i, links in enumerate(controlled_links): if not links:
        continue
    lane_id = links[0][0]

    if "N" in lane_id: lane_map["N"].append(i) elif "S"
    in lane_id: lane_map["S"].append(i) elif "E" in
    lane_id: lane_map["E"].append(i)
```

```
    elif "W" in lane_id: lane_map["W"].append(i)

print("✅ Lane Mapping:", lane_map)

                                #
========================= #
GLOBAL STATE
# =========================

manual_mode = False
lane_state = ""

# ========================= # RL
SETUP
# ========================= Q =
{}
ACTIONS = [0, 1]

alpha = 0.1
gamma = 0.9
epsilon = 0.05

last_action = 0
action_timer = 0
                                MIN_PHASE_TIME = 10

# ========================= #
METRICS
# =========================
```

```
total_wait_history = []

                                    #
========================= # RL
FUNCTIONS
# =========================

def get_state():
    lanes = traci.lane.getIDList()


    N = sum(traci.lane.getLastStepVehicleNumber(l) for l in lanes if "N"
            in l) in l)
    S = sum(traci.lane.getLastStepVehicleNumber(l) for l in lanes if "S" E =


    sum(traci.lane.getLastStepVehicleNumber(l) for l in lanes if
                                        E" in l)
    W = sum(traci.lane.getLastStepVehicleNumber(l) for l in lanes if "W" in l)


    state = (min(N//5,10), min(S//5,10), min(E//5,10), min(W//5,10)) total =
    N+S+E+W


    return state, total


def choose_action(state):
    if random.random() < epsilon: return
        random.choice(ACTIONS)


    if state not in Q: Q[state]
        = [0,0]
```

```
    return Q[state].index(max(Q[state]))

def update_q(state, action, reward, next_state): if state not in
    Q:
        Q[state] = [0,0]
    if next_state not in Q: Q[next_state] =
        [0,0]

    Q[state][action] += alpha * (
        reward + gamma * max(Q[next_state]) - Q[state][action]
    )

                                #
========================= #
SIGNAL CONTROL
# =========================
def apply_action(action):
    current = traci.trafficlight.getRedYellowGreenState(tls_id) length
    = len(current)

    state = ["r"] * length mid
    = length // 2

    if action == 0:
        for i in range(mid):
            state[i] = "G"
    else:
        for i in range(mid, length):
```

```
        state[i] = "G" traci.trafficlight.setRedYellowGreenState(tls_id, "".join(state))


                                #
========================= # SIMULATION
LOOP
# =========================
def simulation_loop():
      global manual_mode, lane_state, last_action, action_timer

   while True: try:
        if traci.simulation.getMinExpectedNumber() > 0: traci.simulationStep()

                                                if manual_mode and lane_state:
                                                   traci.trafficlight.setRedYe
                                                   llowGreenState(tls_id)
```

```
    else:
        state, total = get_state()

        total_wait_history.append(total) if
        len(total_wait_history) > 100:
            total_wait_history.pop(0)

        if action_timer < MIN_PHASE_TIME: action =
            last_action
            action_timer += 1
        else:
            action = choose_action(state)
            last_action = action action_timer = 0

        apply_action(action)

        for _ in range(5): traci.simulationStep()

        next_state, next_total = get_state() reward
        = -next_total

        update_q(state, action, reward, next_state)

else:
    traci.load(["-c", "sumo_config.sumocfg"]) time.sleep(0.02)
```

```
        except Exception as e:
            print(" Simulation error:", e) break

threading.Thread(target=simulation_loop,  daemon=True).start()#
========================= # APIs #
=========================

@app.get("/status") def
status():
    state, total = get_state()
    return {"total": total, "mode": "Manual" if manual_mode else "RL-
Auto"}


@app.get("/metrics") def
metrics():
    if not total_wait_history: return
        {"avg_wait": 0}
    return {"avg_wait":
sum(total_wait_history)/len(total_wait_history)}


@app.post("/set_lane") def
set_lane(data: dict):
    global manual_mode, lane_state


    manual_mode = True direction
    = data.get("dir")


    current = traci.trafficlight.getRedYellowGreenState(tls_id) state = ["r"] *
    len(current)
```

```
    for idx in lane_map.get(direction, []): state[idx] =
        "G"
    lane_state = "".join(state) return
    {"status": "ok"}

@app.post("/auto_mode") def
auto_mode():
        global manual_mode manual_mode
    = False
    return {"status": "RL active"}

@app.post("/emergency") def
emergency(data: dict):
    direction = data.get("dir")

    current = traci.trafficlight.getRedYellowGreenState(tls_id) state = ["r"] *
    len(current)

    for idx in lane_map.get(direction, []): state[idx] =
        "G"

    traci.trafficlight.setRedYellowGreenState(tls_id, "".join(state)) return
    {"status": "emergency"
```

REFERENCES

1. Bazzan, ALC & Klügl, F 2014, ‘A review on agent-based technology for traffic and transportation’, The Knowledge Engineering Review, vol. 29, no. 3, pp. 375–403.
2. Busoniu, L, Babuska, R & De Schutter, B 2008, ‘A comprehensive survey of multiagent reinforcement learning’, IEEE Transactions on Systems, Man, and Cybernetics — Part C: Applications and Reviews, vol. 38, no. 2, pp. 156–172.
3. Li, L, Lv, Y & Wang, FY 2016, ‘Traffic signal timing via deep reinforcement learning’, IEEE/CAA Journal of Automatica Sinica, vol. 3, no. 3, pp. 247–254.
4. Liang, X, Du, X, Wang, G & Han, Z 2019, ‘A deep reinforcement learning network for traffic light cycle control’, IEEE Transactions on Vehicular Technology, vol. 68, no. 2, pp. 1243–1253.
5. Redmon, J & Farhadi, A 2018, ‘YOLOv3: An incremental improvement’, arXiv preprint, arXiv:1804.02767.
6. Praveen, S, Karthik, R & Anand Kumar 2021, ‘Vehicle detection and counting using YOLO for smart traffic management’, in Proceedings of the IEEE International Conference on Intelligent Systems, IEEE, pp. 1–6.
7. Kumar, S & Sharma, A 2022, ‘Smart traffic control system using machine learning and IoT’, in Proceedings of the IEEE International Conference on Smart Cities, IEEE, pp. 1–6.
8. Li, Z, Nourinejad, M & Kamel, M 2020, ‘Real-time traffic signal control with deep Q-network’, IEEE Transactions on Intelligent Transportation Systems, vol. 21, no. 2, pp. 590–602.
9. Dong, NH, Ding, Z & Li, SE 2019, ‘Adaptive traffic signal control based on reinforcement learning’, IEEE Transactions on Vehicular Technology, vol. 68, no. 12, pp. 12241–12253.
10. Wei, W, Li, Y, Ma, H & Cheng, J 2019, ‘Intelligent traffic signal control using reinforcement learning’, IEEE Access, vol. 7, pp. 40797–40809.